ASSIGNMENT-16.1

Name: R.Shashideepika

H NO:2303A51871

Batch-13

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

o Insert a new student record.

o Fetch all courses enrolled by a student.

o Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student– course relationships.

Definitions:

Primary Key: A Primary Key uniquely identifies each record in a table.

Foreign Key: A Foreign Key is used to link tables together.

3. Relationships Between Tables

Type of Relationship:

Many-to-Many (M:N) between Students and Courses

- One student can enroll in many courses
- One course can have many students

• **Generate queries:**

o Insert a new student record.

o Fetch all courses enrolled by a student.

o Count number of students in each course.

Query:

```sql
-- Insert Enrollments
INSERT INTO Enrollments (StudentID, CourseID)
VALUES (1, 1);

INSERT INTO Enrollments (StudentID, CourseID)
VALUES (1, 2);

INSERT INTO Enrollments (StudentID, CourseID)
VALUES (2, 1);

-- ========================================
-- QUERIES
-- ========================================

-- 1. Insert a New Student Record
INSERT INTO Students (FirstName, LastName, DateOfBirth)
VALUES ('Bob', 'Brown', '2002-03-15');

-- 2. Fetch All Courses Enrolled by a Student (StudentID = 1)
SELECT c.CourseName, c.Credits
FROM Enrollments e
JOIN Courses c ON e.CourseID = c.CourseID
WHERE e.StudentID = 1;

-- 3. Count Number of Students in Each Course
SELECT c.CourseName, COUNT(e.StudentID) AS NumberOfStudents
FROM Courses c
LEFT JOIN Enrollments e ON c.CourseID = e.CourseID
GROUP BY c.CourseID;
Select * from Students;
```

Output:

SQL ▾ 1 / 1 1 - 3 of 3

StudentID	FirstName	LastName	DateOfBirth
1	John	Doe	2000-01-01
2	Alice	Smith	2001-05-10
3	Bob	Brown	2002-03-15

Justification: The relationship between the *Students* and *Courses* tables is a many-to-many relationship, because a single student can enroll in multiple courses, and each course can have multiple students. This relationship cannot be represented directly in a single table, so it is implemented using the Enrollments table, which acts as a junction table.

SQL ▾ 1 / 1 1 - 2 of 2

CourseName	Credits
Database Systems	4
Operating Systems	3

SQL ▾ 1 / 1 1 - 2 of 2

CourseName	NumberOfStudents
Database Systems	2
Operating Systems	1

SQL ▾ 1 / 1 1 - 3 of 3

StudentID	FirstName	LastName	DateOfBirth
1	John	Doe	2000-01-01
2	Alice	Smith	2001-05-10
3	Bob	Brown	2002-03-15

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

o List all appointments for a specific doctor.

o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

Queries

```sql
-- Insert Doctors
INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. Smith', 'Cardiology');
INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. John', 'Neurology');
INSERT INTO Doctors (Name, Specialization) VALUES ('Dr. Emily', 'Pediatrics');

-- Insert Patients
INSERT INTO Patients (Name, DateOfBirth) VALUES ('Alice', '1995-06-15');
INSERT INTO Patients (Name, DateOfBirth) VALUES ('Bob', '1990-03-20');
INSERT INTO Patients (Name, DateOfBirth) VALUES ('Charlie', '1985-12-05');

-- Insert Appointments
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (1, 1, '2025-03-01', 'Heart Checkup');

INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (1, 2, '2025-03-05', 'BP Issue');
--  Query 1. List all appointments for a specific doctor
INSERT INTO Appointments (DoctorID, PatientID, AppointmentDate, Diagnosis)
VALUES (2, 1, '2025-03-10', 'Migraine');
SELECT d.Name AS DoctorName, p.Name AS PatientName, a.AppointmentDate, a.Diagnosis
FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Patients p ON a.PatientID = p.PatientID
WHERE d.DoctorID = 1;
--  Query 2. Retrieve patient history by patient ID
SELECT p.Name AS PatientName, d.Name AS DoctorName, a.AppointmentDate, a.Diagnosis
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
WHERE p.PatientID = 1;
--  Query 3. Count total patients treated by each doctor
SELECT d.Name AS DoctorName, COUNT(DISTINCT a.PatientID) AS TotalPatients
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID;
```

Output:

DoctorName	PatientName	AppointmentDate	Diagnosis
Dr. Smith	Alice	2025-03-01	Heart Checkup
Dr. Smith	Bob	2025-03-05	BP Issue

SQL ▾ 1 / 1 1 - 2 of 2

PatientName	DoctorName	AppointmentDate	Diagnosis
Alice	Dr. Smith	2025-03-01	Heart Checkup
Alice	Dr. John	2025-03-10	Migraine

SQL ▾ 1 / 1 1 - 3 of 3

DoctorName	TotalPatients
Dr. Smith	2
Dr. John	1
Dr. Emily	0

Ln 59, Col 52 Spaces: 4 UTF-8 CRLF { } SQL SQLite: :memory: Go Live

Justification: The Appointments table links doctors and patients via foreign key. Constraints like NOT NULL, UNIQUE, and valid date checks ensure data integrity, enabling efficient storage, retrieval, and querying of hospital data.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

Queries

```sql
-- Books
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('DBMS', 'Author A', 5);
INSERT INTO Books (Title, Author, AvailableCopies) VALUES ('OS', 'Author B', 3);

-- Members
INSERT INTO Members (Name, JoinDate) VALUES ('Alice', '2024-01-01');
INSERT INTO Members (Name, JoinDate) VALUES ('Bob', '2024-02-01');

-- Loans
INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 1, '2025-02-01', NULL); -- currently issued

INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (2, 2, '2025-01-01', NULL); -- overdue

INSERT INTO Loans (BookID, MemberID, LoanDate, ReturnDate)
VALUES (1, 2, '2025-03-01', '2025-03-10'); -- returned
-- Query 1: Retrieve all books currently issued
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
-- Query 2: Find overdue books (loan date > 30 days)
SELECT b.Title, m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL
AND DATE(l.LoanDate) <= DATE('now', '-30 days');
-- Query 3: Count number of books loaned by each member
SELECT m.Name AS MemberName, COUNT(l.BookID) AS TotalBooksLoaned
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Output

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ 1 / 1 1 - 2 of 2

Title	MemberName	LoanDate
DBMS	Alice	2025-02-01
OS	Bob	2025-01-01

SQL ▾ 1 / 1 1 - 2 of 2

MemberName	TotalBooksLoaned
Alice	1
Bob	2

Justification: The library database is designed using three normalized tables: Books, Members, and Loans. Each table has a primary key to uniquely identify records, while the Loans table uses foreign keys to establish relationships between books and members.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

Queries

```sql
-- Users
INSERT INTO Users (Name, Email) VALUES ('Alice', 'alice@gmail.com');
INSERT INTO Users (Name, Email) VALUES ('Bob', 'bob@gmail.com');

-- Products
INSERT INTO Products (ProductName, Price) VALUES ('Laptop', 50000);
INSERT INTO Products (ProductName, Price) VALUES ('Phone', 20000);

-- Orders
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-01');
INSERT INTO Orders (UserID, OrderDate) VALUES (1, '2025-03-10');
INSERT INTO Orders (UserID, OrderDate) VALUES (2, '2025-03-15');

-- OrderDetails
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 1, 1);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (1, 2, 2);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (2, 2, 1);
INSERT INTO OrderDetails (OrderID, ProductID, Quantity) VALUES (3, 1, 1);
-- Query 1: Retrieve all orders by a user
SELECT u.Name, o.OrderID, o.OrderDate
FROM Orders o
JOIN Users u ON o.UserID = u.UserID
WHERE u.UserID = 1;
-- Query 2: Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID
ORDER BY TotalSold DESC
LIMIT 1;
-- Query 3: Calculate total revenue in a given month
SELECT SUM(p.Price * od.Quantity) AS TotalRevenue
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Orders o ON od.OrderID = o.OrderID
WHERE strftime('%Y-%m', o.OrderDate) = '2025-03';
```

Output

Name	OrderID	OrderDate
Alice	1	2025-03-01
Alice	2	2025-03-10

SQL ▾ 1 / 1 1 - 1 of 1

ProductName	TotalSold
Phone	3

SQL ▾ 1 / 1 1 - 1 of 1

TotalRevenue
160000.0

Ln 67, Col 50 (2292 selected) Spaces: 4 UTF-8 CRLF { } SQL SQLite: :memory: (•) Go Live

Justification: The database is structured in a normalized form to eliminate redundancy and ensure data integrity. The Users table stores customer details, while the Products table maintains product information independently. Orders are separated into a different table to record each transaction, and OrderDetails acts as a junction table to handle the many-to-many relationship between orders and products.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

Queries

```sql
-- Students
INSERT INTO Students (Name) VALUES ('Alice');
INSERT INTO Students (Name) VALUES ('Bob');

-- Subjects
INSERT INTO Subjects (SubjectName) VALUES ('Math');
INSERT INTO Subjects (SubjectName) VALUES ('Science');

-- Exams
INSERT INTO Exams (SubjectID, ExamDate) VALUES (1, '2025-03-01');
INSERT INTO Exams (SubjectID, ExamDate) VALUES (2, '2025-03-05');

-- Results
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 1, 85);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (1, 2, 90);
INSERT INTO Results (StudentID, ExamID, Marks) VALUES (2, 1, 75);
-- Query 1: Insert examination result for a student
INSERT INTO Results (StudentID, ExamID, Marks)
VALUES (2, 2, 88);
-- Query 2: Retrieve subject-wise marks for a student
SELECT s.SubjectName, r.Marks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
WHERE r.StudentID = 1;
-- Query 3: Calculate average marks for each subject
SELECT s.SubjectName, AVG(r.Marks) AS AverageMarks
FROM Results r
JOIN Exams e ON r.ExamID = e.ExamID
JOIN Subjects s ON e.SubjectID = s.SubjectID
GROUP BY s.SubjectID;
```

Output:

SubjectName	Marks
Math	85
Science	90

SQL ▾ 1 / 1 1 - 2 of 2

SubjectName	AverageMarks
Math	80.0
Science	89.0

SQL ▾ 1 / 1 1 - 4 of 4

ResultID	StudentID	ExamID	Marks
1	1	1	85
2	1	2	90
3	2	1	75
4	2	2	88

Justification: The database is designed using four normalized tables: Students, Subjects, Exams, and Results. Each table has a primary key to uniquely identify records, while foreign keys establish relationships between them. The Exams table links subjects with exam dates, and the Results table acts as a junction table connecting students and exams, enabling a many-to-many relationship. Referential integrity is maintained through foreign key constraints, ensuring that results cannot exist without valid students and exams.